

Deep Learning Classification

UrbanSound8K Dataset

Henrique Teixeira
João Ferreira
Leonor Carvalho

Work Objectives

In this project, we focus on:

- Develop deep learning models for urban sound classification.
- Implement two different classifiers: CNN and RNN.
- Preprocess and prepare audio data for model training.
- Evaluate performance using accuracy and confusion matrices.
- Aim for robust and generalizable classifiers.

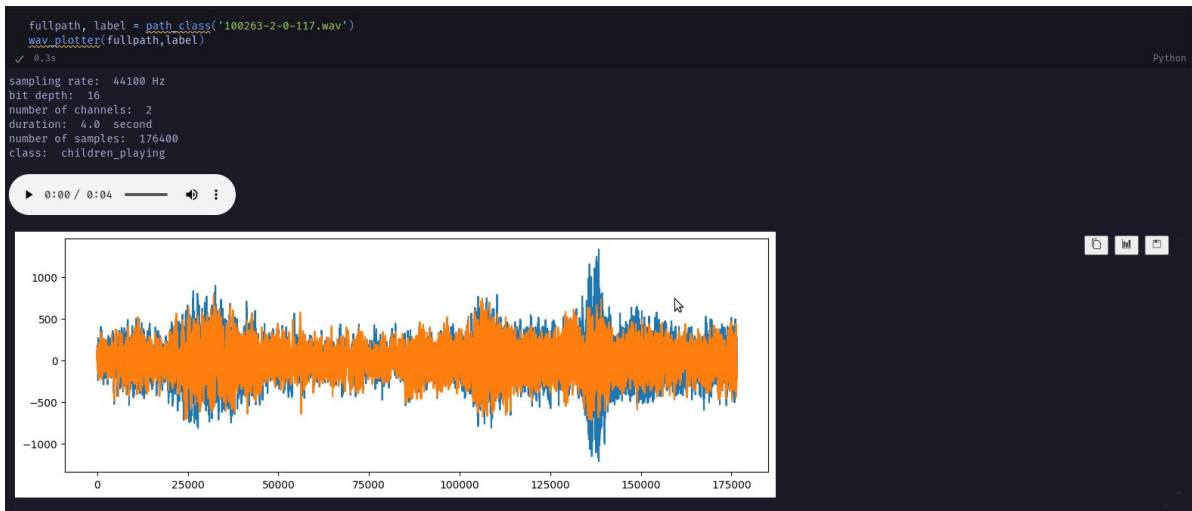
UrbanSound8k dataset

The UrbanSound8K dataset, a widely used benchmark that contains 8732 labeled audio excerpts spanning ten everyday urban sound classes, including car horn, dog bark, siren, engine idling, jackhammer, among others. Each sound clip is no longer than four seconds, presenting challenges related to signal variability, noise, and differences in duration or sampling rate.

UrbanSound8k dataset - Data Exploration

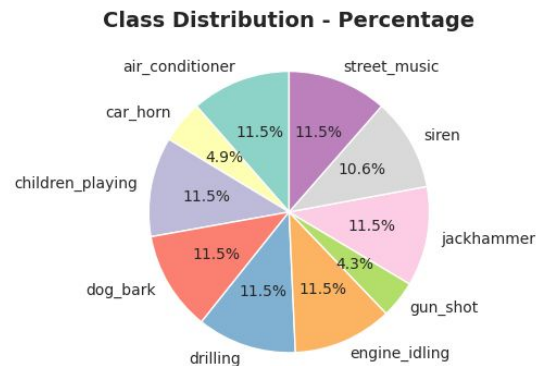
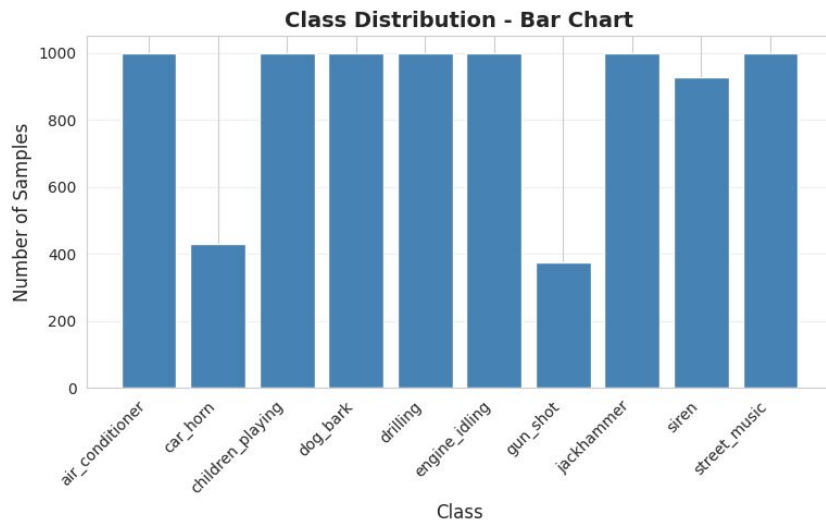
We started by analysing the dataset structure and some of the files.

	slice_file_name	fsID	start	end	salience	fold	classID	class
0	100032-3-0-0.wav	100032	0.0	0.317551	1	5	3	dog_bark
1	100263-2-0-117.wav	100263	58.5	62.500000	1	5	2	children_playing
2	100263-2-0-121.wav	100263	60.5	64.500000	1	5	2	children_playing
3	100263-2-0-126.wav	100263	63.0	67.000000	1	5	2	children_playing
4	100263-2-0-137.wav	100263	68.5	72.500000	1	5	2	children_playing

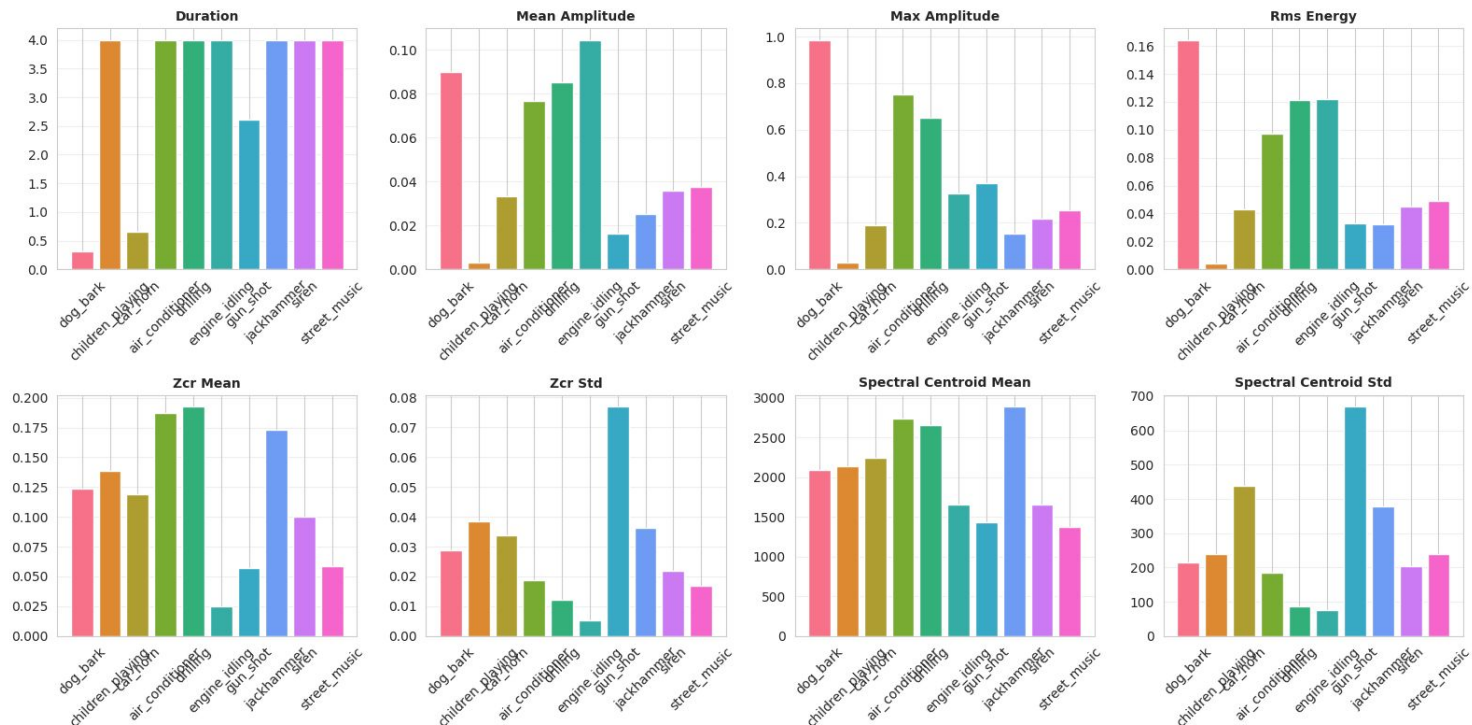


UrbanSound8k dataset - Data Exploration

We identified class imbalance within the dataset

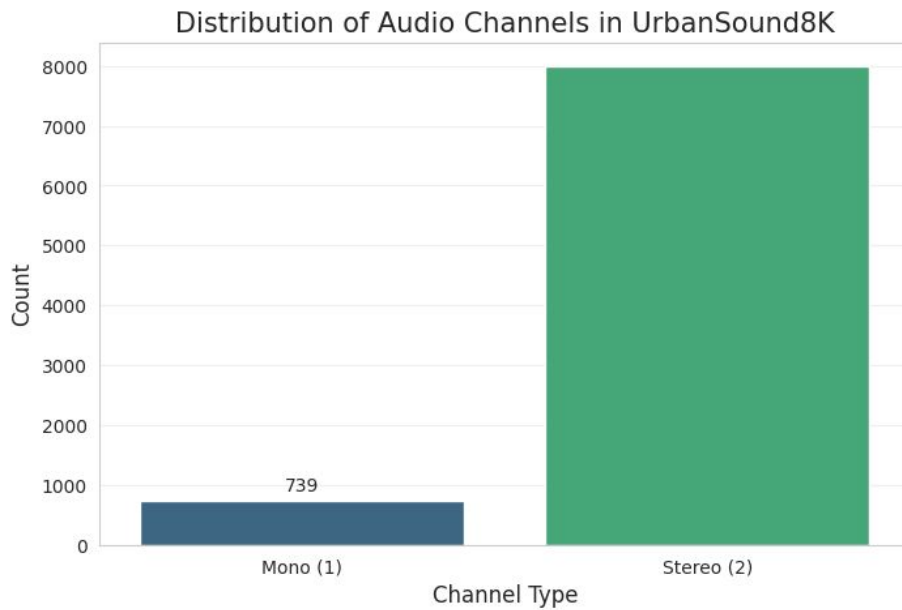


Dealing with class imbalance



Dealing with class imbalance

We also identified with mono/stereo class imbalance



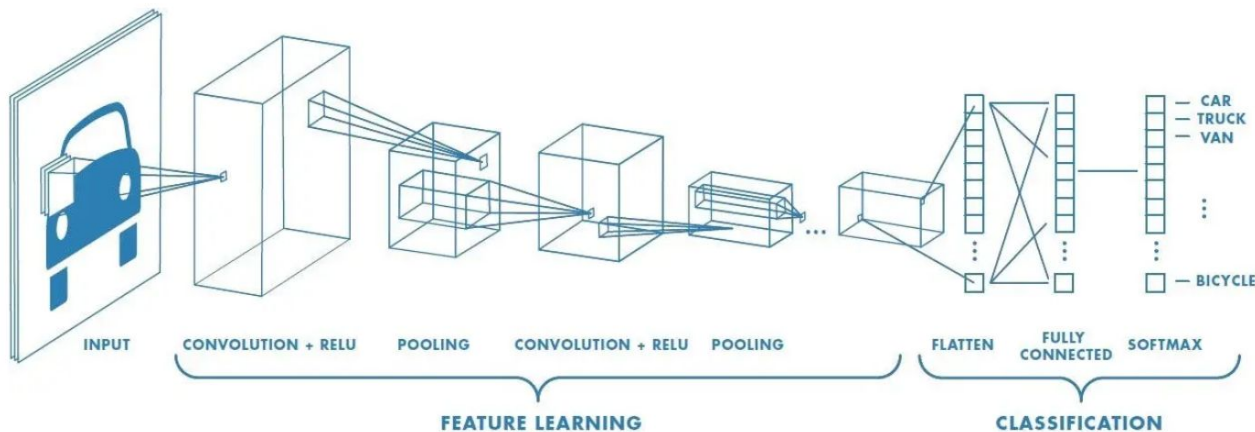
Dealing with class imbalance

- Applied data augmentation on the minority classes in the training folds
- Converted all audio channels to mono

Classifiers

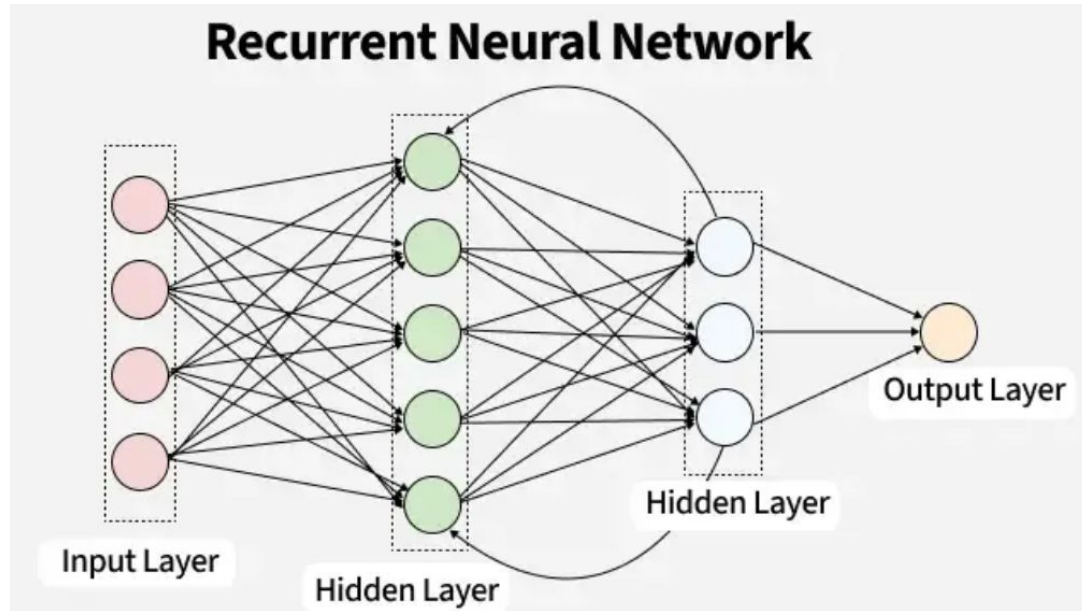
Convolutional Neural Network

A convolutional neural network (CNN) is a type of deep learning model designed to process data with a grid-like structure, most commonly images and videos



Recurrent Neural Networks

A Recurrent Neural Network (RNN) is a type of deep learning model that processes sequential data, such as text or time series, by using "memory" to remember past information and use it to influence future predictions

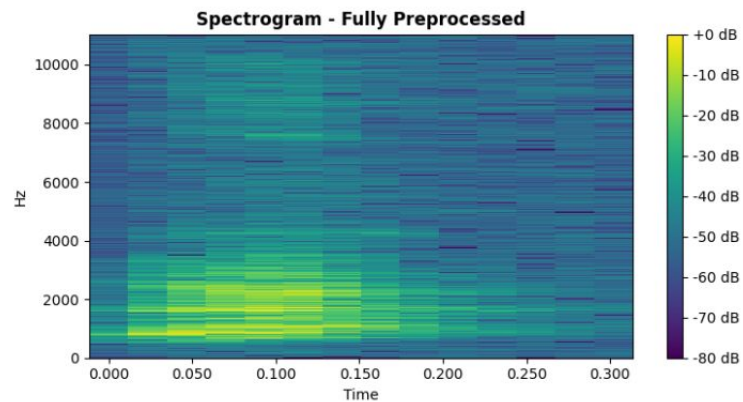
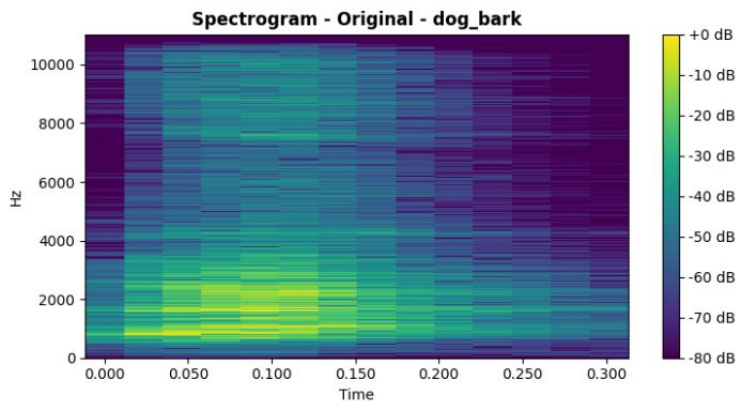
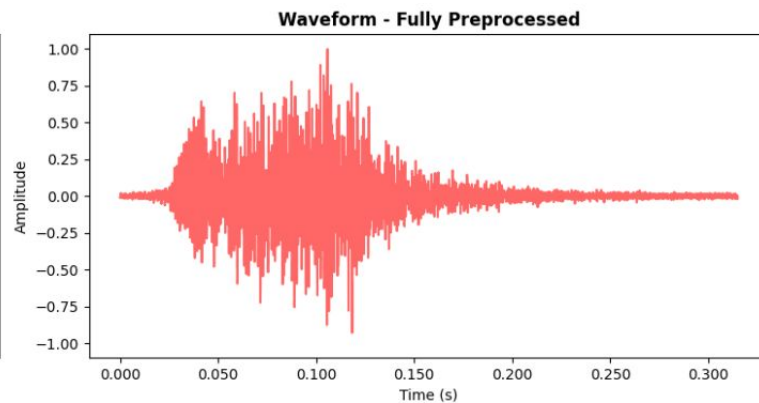
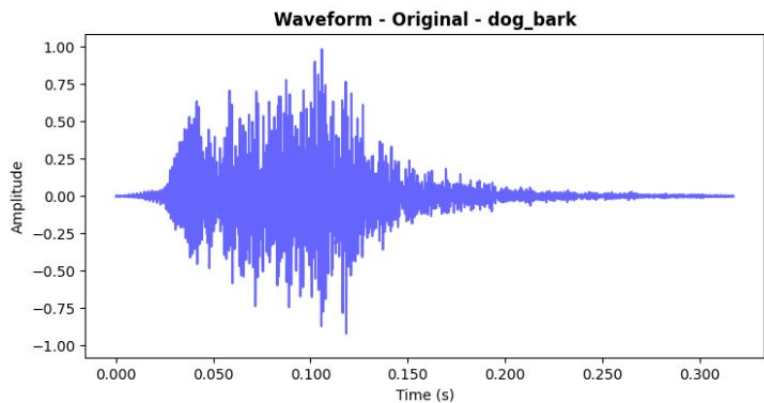


Artificial Data Augmentation

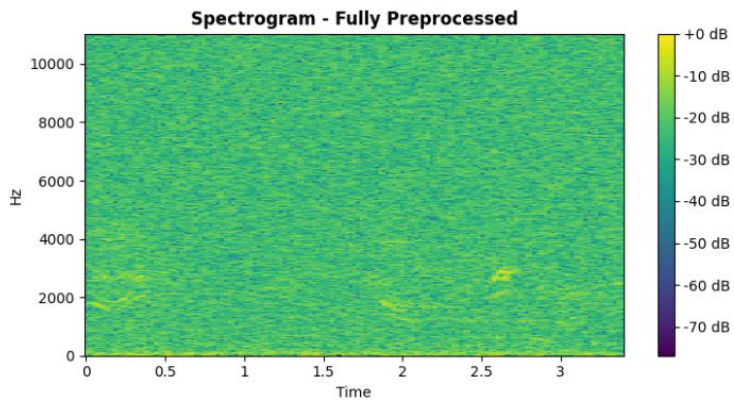
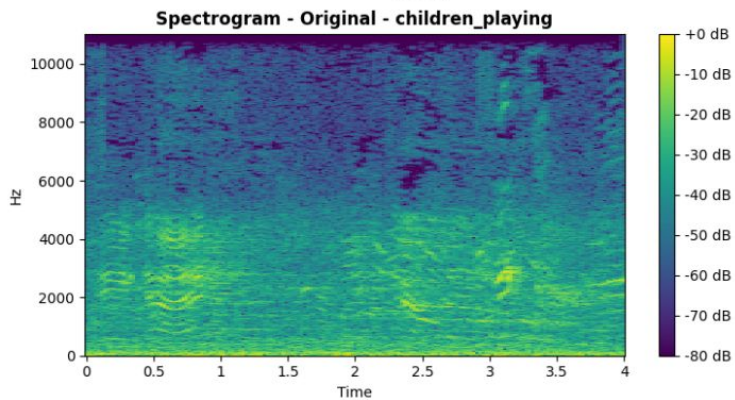
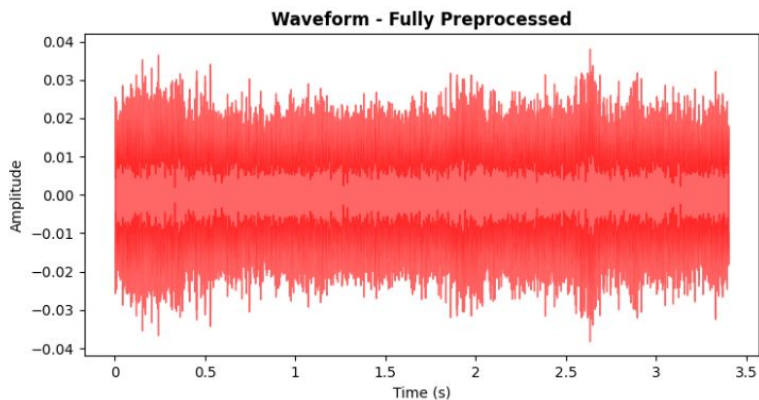
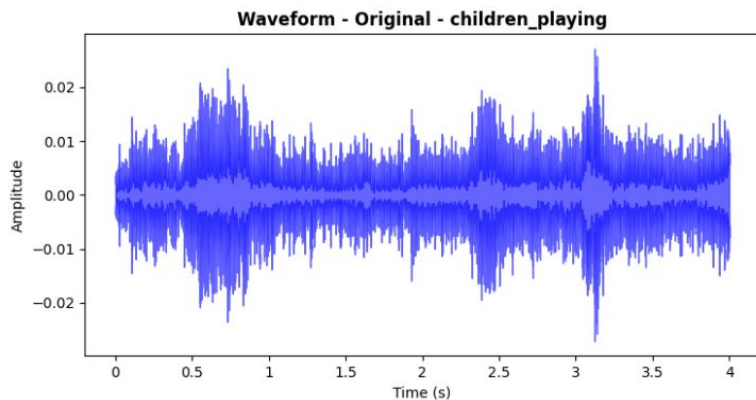
This makes the model more robust:

- Adding noise to the dataset
- Randomly cropping the sound
- Pitch shifting
- Time stretch

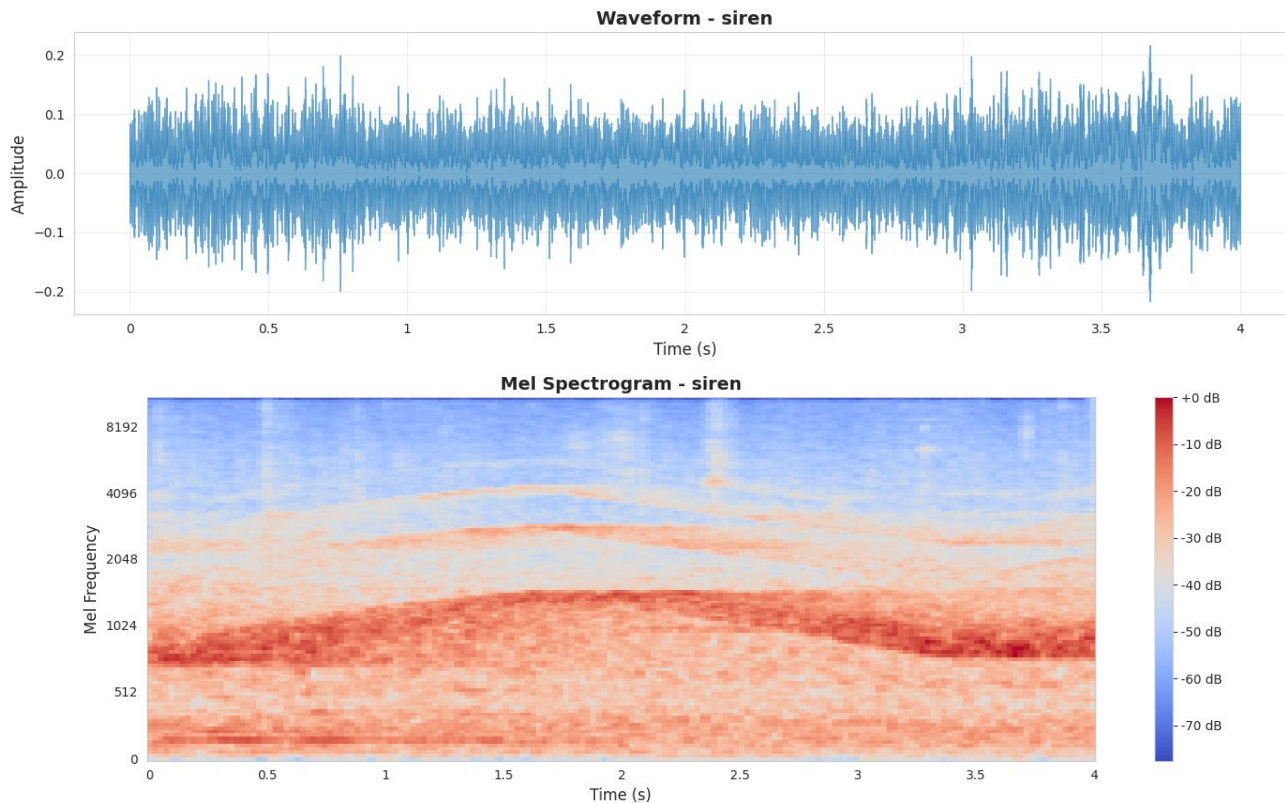
After Preprocessing (example 1)



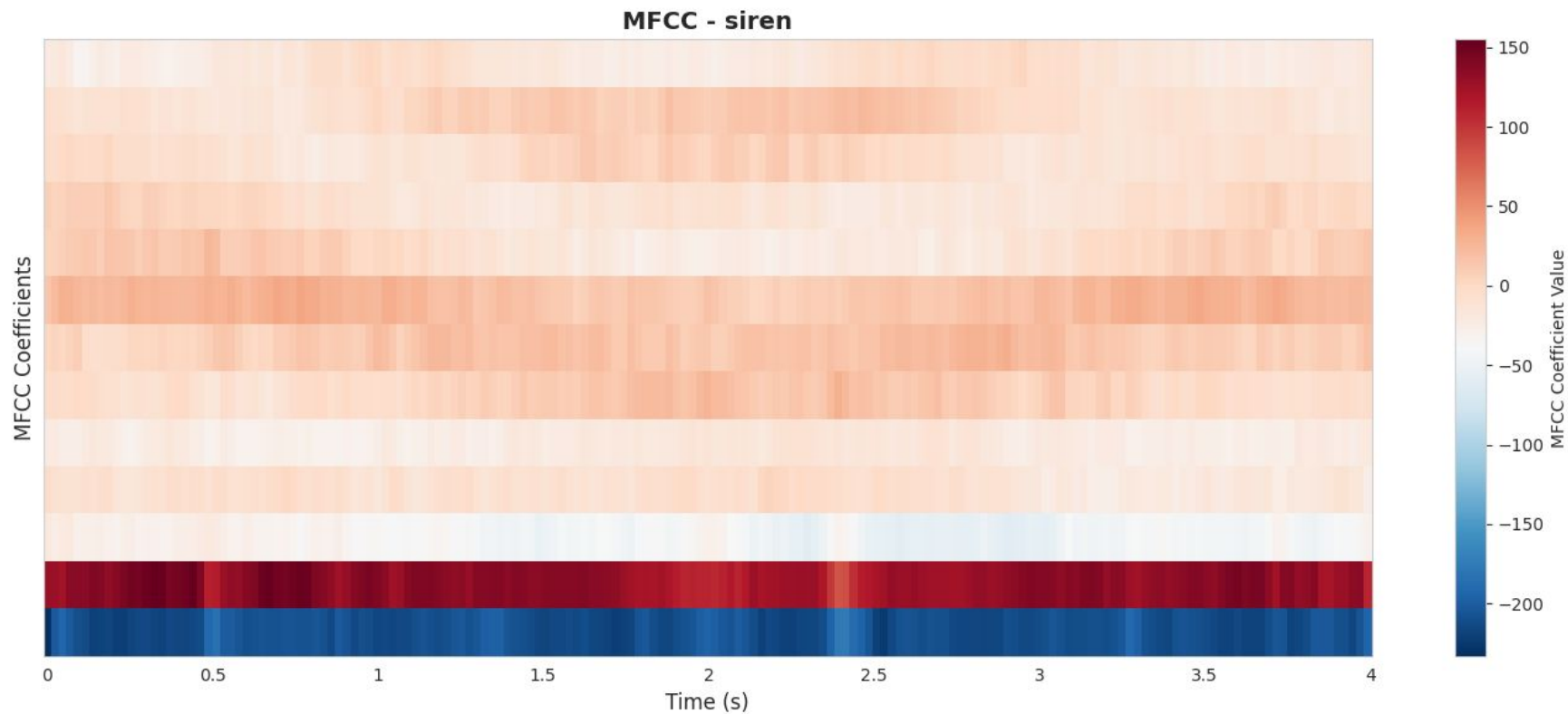
After Preprocessing (example 2)



Processing Data - Mel Spectrograms



Processing Data - MFCC



Decisions - CNN

What Optimizer to use? - Adam

What Loss to use? - Cross Entropy Loss

How many Layers in the NN? - 5

How to deal with the learning rate? - LR Scheduler (starting at 0.001)

How to deal with overfitting? - Dropouts + L2 regularization (weight decay)

What to feed the NN? MelSpectrogram

Hyper Parameters

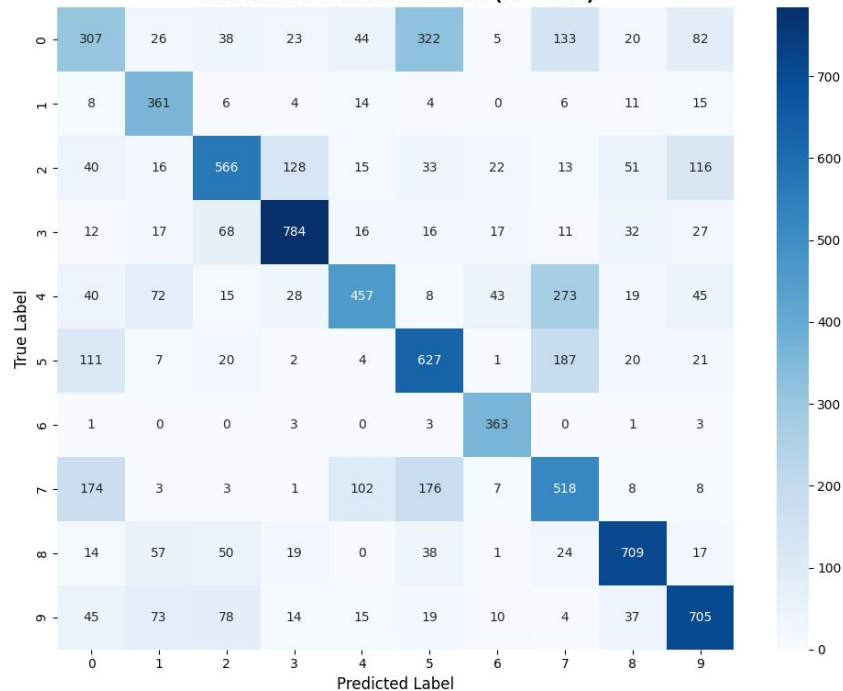
We tested various configurations in the beginning and the following stood out:

- Optimizer: Adam vs SGD, Adam performed the best by 5%
- Criterion: Cross Entropy Loss vs NLL, Cross Entropy Loss performed the best by 5%
- Mel Spectrogram vs MFCC, not a large difference, however, Mel Spectrogram performed the best

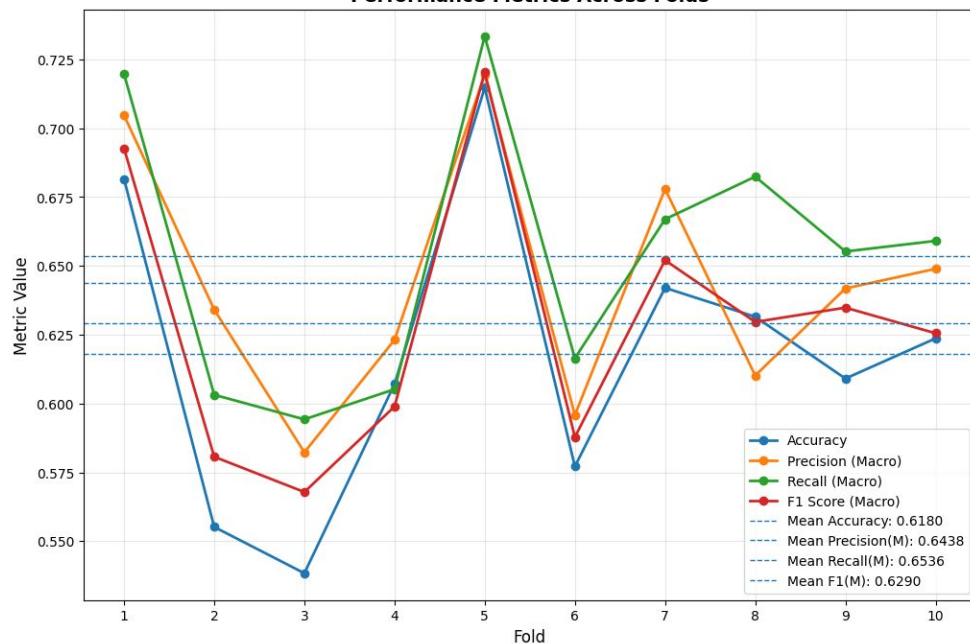
In the notebook, the configurations ran are the best.

Results - CNN

Cumulative Confusion Matrix (10 Folds)



Performance Metrics Across Folds



Testing with Deep Fool - CNN

DEEPOOOL RESULTS (SAMPLED DATA)

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#)

	fold	samples_tested	total_correct	fool_rate	mean_perturb_norm
0	1	100	738	1.0	3.475612
1	3	100	638	1.0	3.799358
2	5	100	669	1.0	4.513592

== Averages Across Tested Folds ==

Average Fool Rate: 1.0000

Average Perturbation Norm: 3.9295

Total Samples Tested: 300

Decisions - RNN

What Optimizer to use? - Adam

What Loss to use? - Cross Entropy Loss

How many Layers in the NN? - 3 LSTM + 1 Dense Layer

How to deal with the learning rate? - LR Scheduler (starting at 0.0035)

How to deal with overfitting? - Dropouts + L2 regularizator (weight decay)

What to Feed the NN? MelSpectrogram

Hyper Parameters

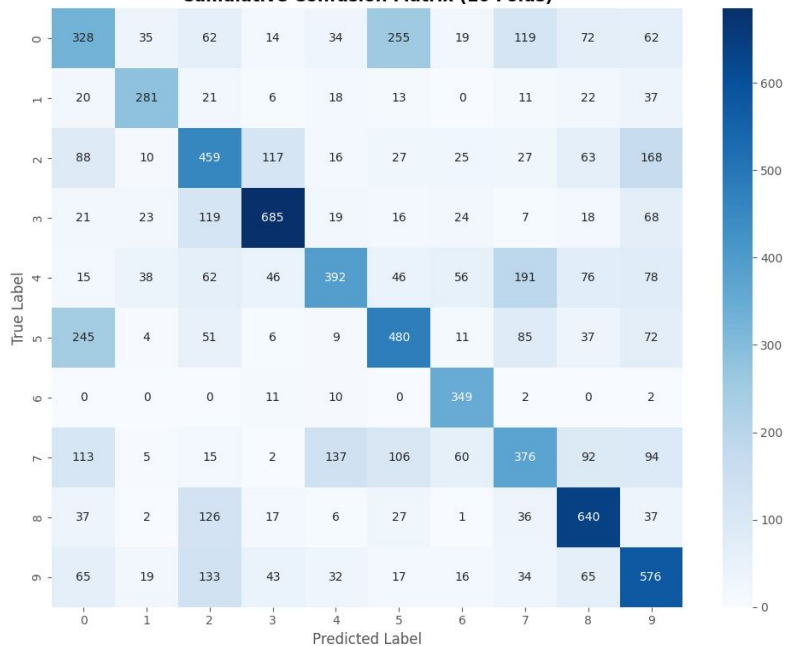
We tested various configurations in the beginning and the following stood out:

- Optimizer: Adam vs SGD, Adam performed the best by 10%
- Criterion: Cross Entropy Loss vs NLL, Cross Entropy Loss performed the best by 5%
- Mel Spectrogram vs MFCC, Mel Spectrogram performed the best by 5%

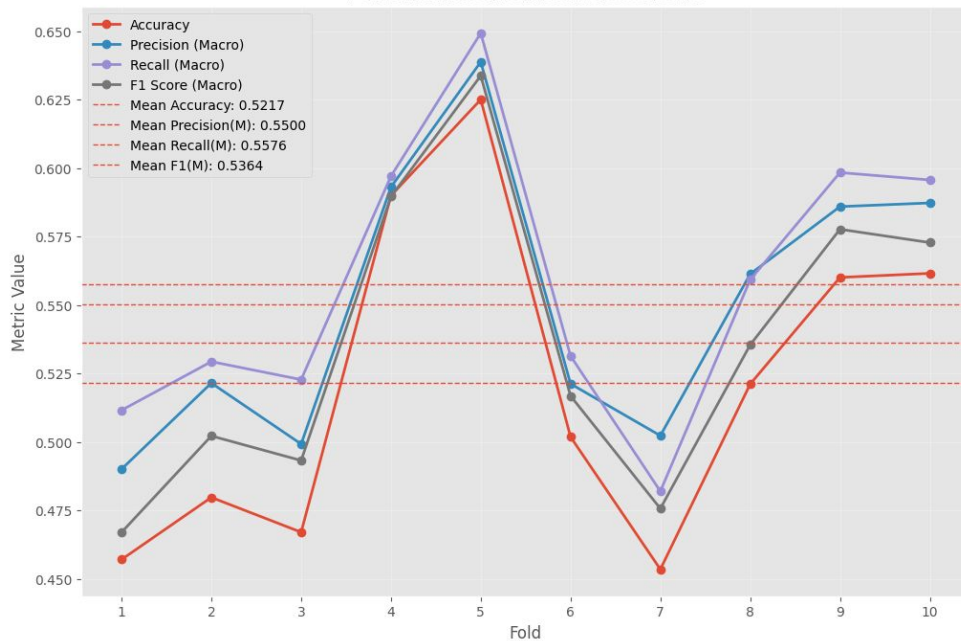
Our RNN was overfitting in all our initial configurations, so we did hyperparameter tuning to find a stable configuration.

Results - RNN

Cumulative Confusion Matrix (10 Folds)



Performance Metrics Across Folds



Testing with Deep Fool - RNN

DEEPOOL RESULTS (SAMPLED DATA)

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#)

	fold	samples_tested	total_correct	fool_rate	mean_perturb_norm
0	1	100	684	1.0	9.807635
1	3	100	646	1.0	41.557846
2	5	100	585	1.0	71.825453

== Averages Across Tested Folds ==

Average Fool Rate: 1.0000

Average Perturbation Norm: 41.0636

Total Samples Tested: 300

Conclusion and Future Work

- Both model performance obtained could've been better
- Deep fool analysis demonstrated the clear weakness in the training and lack of robustness, even though the data preparation with augmentation